

Practice - OneCompiler.com and Google Cloud Shell Terminal Tools

Introduction to Programming II - CS 101 - Catholic Polytechnic University - Spring 2026

C++ Programming with OneCompiler

Objective

The goal of this tutorial is to introduce students to writing, compiling, and running simple C++ programs using OneCompiler.com.

This is important so that you can complete your assignments & follow along with me in class while coding ("code along") if you wish.

Getting Started with OneCompiler

1. Open your web browser and go to <https://onecompiler.com/cpp>
2. Click on "New Program" and select "C" as the programming language.
3. You will see a default template with a simple C program.

Creating a new C++ file in OneCompiler

OneCompiler has two organisational levels: 1) code, 2) file, and we'll change them both.

1. Delete the existing code to start with your own program.
2. Click on the pen in the middle of the page to change the name of the code to "Hello World" and add a short description:

First program with OneCompiler. CSC 100 class practice: 01/17/2025.

3. Add the tag helloworld. Tags help greatly with search + find.
4. Check the Visibility, change it to Unlisted (People with the Link), and click on Save.
5. Open the main menu (three horizontal lines, upper left side) and choose My Account.
6. Click on CODES. You should see the last code you edited. If you use dark mode (button at top of the page), you won't see the tag. In the settings (three dots) you can change the visibility or delete the code.
7. Click on the name of your code to get back to the editor view.
8. Hovering over the filename NewFile1.cpp, find the edit pen: In the popup, enter a new file name: helloworld.cpp.
9. To clean the slate, open the settings (three dots) and choose Clear Output. You can also download your file from here.
10. Open the setting again, and choose Editor Settings. In the popup, check Disable Code Autocomplete/Suggestions.

Basic Structure of a C Program

Every C program has a basic structure:

- Header information (*preprocessor*)
- main program ending with `return 0;`
- Program body enclosed in `{ }`
- Comments (optional) followed by `//`

The minimal version (no output):

```
main() // main function
{      // start main program
}      // end main program
```

C is a 100% subset of C++. Here is the C-style version of `helloworld.cpp` - indentation is optional but useful:

```
#include <iostream> // import input/output functions
int main(void) { // main program begins
    puts("Hello, world!"); // print greeting string
    return 0; // confirm that program finished
} // main program ends
```

Hello, world!

And finally, the full C++ version of `helloworld.cpp`: It now includes a program header that all programs should have.

```
// print "Hello, world!" to stdout (standard output)
#include <iostream> // import input/output functions

int main(void) { // main program begins
    // print string to stdout and end with new line
    std::cout << "Hello, World!" << std::endl;

    return 0; // confirm that program finished
} // main program ends
```

Hello, World!

Your task:

1. Type the program (with comments) into the editor.
2. At the end of each line, press Enter.
3. At the start of a new line, press TAB to indent
4. The file is automatically saved.
5. Click RUN (or CTRL + ENTER) and check the Output field.

Downloading and uploading files

1. Add a program header. Adapt the header below to your own program:

```
/* **** */
/* helloworld.cpp: Print greeting string */
/* Input: none */
```

```

/* Output: Hello, world! */
/* Author: Marcus Birkenkrahe */
/* Date: 1/4/2025 */
/*****/

```

2. After making sure that the program (still) produces the desired output, Download it to your PC using the settings menu (three dots) as helloworld.c.
3. Open Canvas and find the Getting started with OneCompiler assignment.
4. Upload the helloworld.cpp file.
5. Upload the link (URL) to the online file. You find it in the status bar of your browser. Mine is:
<https://onecompiler.com/cpp/44c8gvd6f>

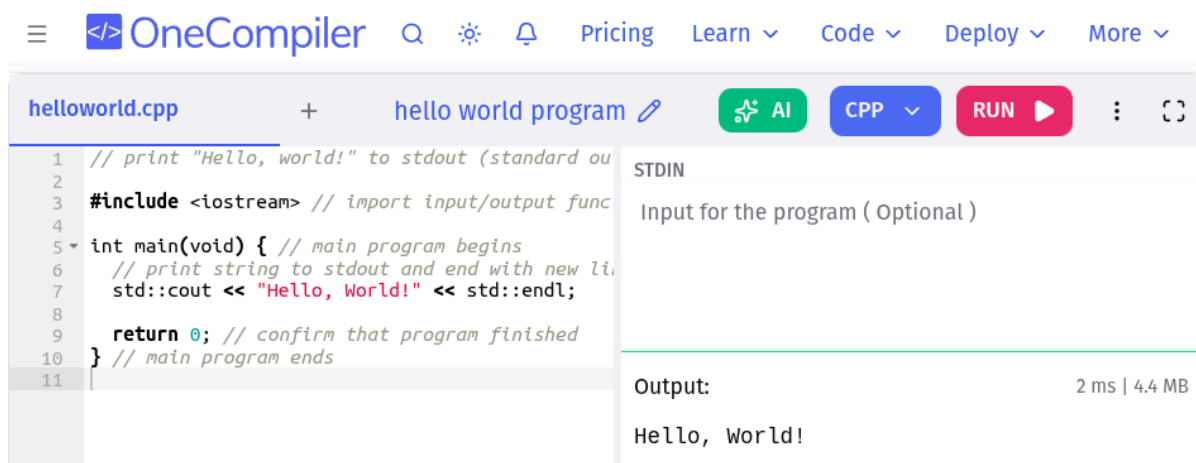


Figure 1: Final result in OneCompiler.com

Extensions

- Below the editor, there is extensive Syntax help for C programming, check it out. You can also get to this with the How to (Wiki) selection in the settings.
- OneCompiler offers a nice free [C tutorial](#) if you want to work ahead. You find it in the top menu (three horizontal lines).
- There are programming [challenges](#) (some of which we'll be doing in and outside of class). You have to pick your language.
- There are [cheatsheets](#), as a useful reference or a condensed overview of an advanced topic - check out C++ Programming language.
- Next time you want to get back straight to the C editor, go to onecompiler.com/c.

C++ Compilation on Google Cloud Shell

Objective

The goal of this tutorial is about using Google Cloud Shell (a Linux VM): I introduce simple Linux commands, and we write, compile, and run a C++ file.

This is important so that you understand what's behind compilation even though on OneCompiler.com, the details are hidden from you.

Getting to the terminal

1. Log into <https://ide.cloud.google.com> - the Linux VM will take a little time (30 sec) to spin up the first time and every time you return to it after a longer break, but it will remain yours with all the files you upload there (I am not sure what the limit is).
2. Instead of working with the Cloud editor, find the tiny terminal sign >_ at the top of the screen to open the terminal. You're in!
3. Don't worry about Gemini, Google's AI though you can use it both in the terminal (start a command with gemini) or in a separate window for help. It's pretty slow and generally inferior to ChatGPT, I find.

Getting around the terminal and repeating commands

1. Commands to try:

```
whoami    # your user name
pwd       # present working directory
ls        # files in pwd [at the start, this shows nothing]
```

2. Make a directory for the class and go there:

```
mkdir cs101    # make a directory
cd cs101      # go there
pwd           # check that you arrived
```

3. We also looked at the built-in history: You can go through the sequence of commands you typed with the up- and the down-arrow, and you can list the 1000 last commands with history. To run a command from that list directly, for example number 503, enter !503 at the prompt.

Writing and saving the source code file

1. Linux has a simple editor, nano that we will use to write our program:

```
nano helloworld.cpp # opens editor on the file `helloworld.cpp`
```

2. In the editor, use CTRL + s to save, CTRL + x to exit. Write the code exactly as follows (contains a syntax mistake):

```
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl
    return 0;
}
```

3. After saving the file, check on the command-line that it exists using the file listing command ls. You can get a lot more information using ls -lt (long, time-ordered file listing).

Compiling source code and running object (machine) code

1. The C/C++ compiler is called gcc / g++ (two commands for the same program). Call g++ directly on your C++ program to generate an executable file:

```
g++ helloworld.cpp
```

Since the file contains a syntax mistake, compilation will abort with this message:

```
Error: expected ';' before 'return'
11 |     std::cout << "Hello, world!" << std::endl
    |                                                     ^
12 |     return 0;
    |     ~~~~~
```

It's pretty clear: you need to add the missing semi-colon! Go back into nano, fix the mistake, save and return to the command-line, then run `g++ helloworld.cpp` again.

2. Now things are peachy: Check your files with `ls` again, and you should see a new file, `a.out`. If you use `ls -l`, you can see that it is an executable. Let's run it.
3. Instead of running it, entering the program name only runs into a problem: enter `a.out` and you get the message:

```
a.out: command not found
```

This is so because the computer cannot see what you see! To find any file, it needs to be in the computer's PATH. Enter:

```
echo $PATH
```

You should get a list of directories (program locations) like this:

```
/home/[your name]/.local/bin:/home/[your name]/. .../usr/local/games:/snap/bin
```

4. A shortcut to adding the program location to the PATH is to use the dot `.` to indicate that the file is in the current working directory (`pwd`). This command will work (if your source file is error-free):

```
./a.out
```

5. You don't need to save anything here, just exit by closing the tab, or by entering `exit` at the prompt in the terminal.

Author: Marcus Birkenkrahe

Created: 2026-01-31 Sat 10:44